UAlg FCT
UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

(Developing reliable code)

Class design by contract

Supplementary readings: (

: chapt. 3), (

: chapter 11)

1

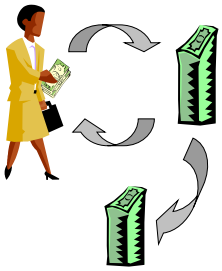
Again

- The test of *class invariants* is a responsibility of the *class*
- The test of *pre conditions* is a responsability of the *client only*!

2

Problem 4: ATM

Develop a program for handling a bank account whose attributes include id, name, and a balance that cannot be negative



3

A class Account

```
/**
 * A simple bank account for which the balance can never be
 * less than zero
 *
 * @inv balance >= 0
 * @inv id is unique and set when account is created
 */

public class Account
{
  private final int id;
  private static int nextAccountId = 0;
  private String name;
  private double balance;
```

4

4

```
/**
 * Initializes an account
 * @param accountName Customer name for account
 * @param initialBalance Initial balance deposited in account
 *
 * @pre true
 * @post getName() == accountName
 * @post getId() == a new number not returned by other accounts
 * @post (initialBalance>=0 AND getBalance()== initialBalance)
 *       OR  getBalance()== 0
 *
 */
public Account(String accountName, double initialBalance) {
      id = nextAccountId++;
      name = accountName;
      if (initialBalance >= 0)
            balance = initialBalance;
      else
            System.exit(0);
}
```

post can be defined using accessors

5

5

```
/**
 * Accessor method to return an int with the account id
 * @pre true
 * @return account id (an int)
 */
public int getId() {
  return id;
}
```

No state changes, no @post

6

```
/**
 * Accessor method to return an int with the account id
 * @return account id (an int)
 */
public int getId() {
  return id;
}
```

No pre-condition, no @pre

7

```
/**
 * Accessor method to return an int with the account id
 */
public int getId() {
  return id;
}
```

In general, this will be enough

8

```
/**
 * Deposit money into the account
 * @param amount The amount to be deposited (double)
 *
 * @pre  amount >= 0
 * @post getBalance()== @pre.getBalance() + amount
 * @return current balance (a double)
 */
public double deposit(double amount)  {

  balance += amount;
  if (balance < 0) System.exit(0); // inv check
  return balance;
}
```

9

Assert for debug only

```
/**
 * Deposit money into the account
 * @param amount The amount to be deposited (double)
 *
 * @pre  amount >= 0
 * @post getBalance() == @pre.getBalance() + amount
 * @return The current balance of the account
 */
public double deposit(double amount)  {
  assert  amount >= 0;
  balance += amount;
  if (balance < 0) Sys
  return balance;
}
```

Assertion checking can be enabled or disabled when launching:

java -ea ClassToRun *enable*
java -da ClassToRun *disable*

10

```
/**
 * Withdraw money from the account
 * @param amount The amount to be withdrawn (double)
 *
 * @pre amount > 0  AND  getBalance() - amount >= 0
* @post getBalance() == @pre.getBalance() - amount
 * @return The current balance of the account (double)
 */
public double withdraw(double amount) {
  assert  amount >= 0;
  assert balance - amount >= 0;

  balance -= amount;

  if (balance < 0) System.exit(0); // inv check

  return balance;

}
```

11

UAlg FCT
UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

An introduction to UML and object-oriented modeling

Readings: ( :chapt. 12)

12

Overview

- Modeling and UML, an Introduction
- Classes and relationship between classes: association, composition, and agregation
- Introduction to UML class and object diagrams
- Notion of *design pattern*
- The Expert pattern

Learning outcomes

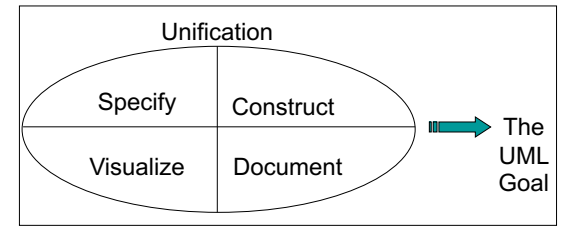
1. Learn how to discover new classes and methods
2. Identify association, aggregation, composition, dependency, and inheritance relationships between classes
3. Describe class relationships using UML class diagrams
4. Apply object-oriented design techniques to building complex programs
5. Understand the notion of design pattern
6. Understand and apply the The Expert pattern

Modeling

- Informally, modeling is the construction of models that explain the characteristics and behaviour of a system
- Modeling involves both a language and a process for using the language to build models
- Modeling first, code next
  (pretty much like in house building: project first before starting to build)

Introducing UML

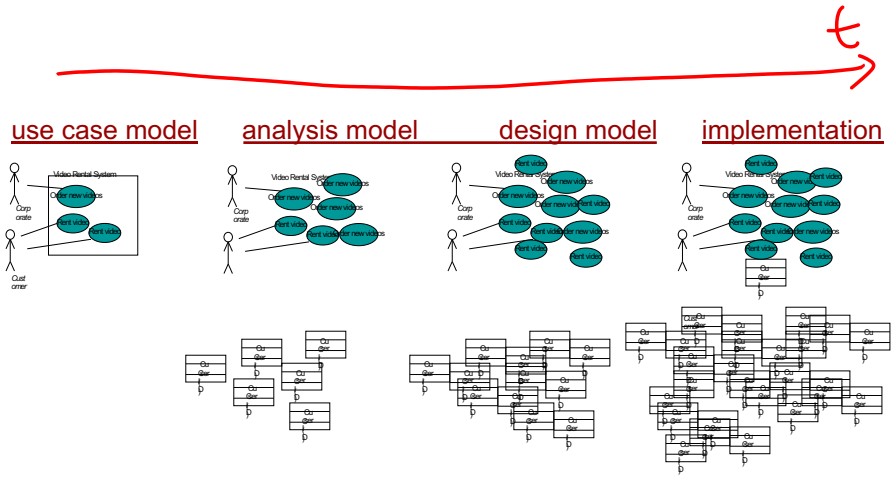
- The Unified Modeling Language (UML) is a modeling language for specifying, visualizing, constructing, and documenting the artifacts of a system-intensive process.



Programming Languages | Tools / Repository Specification | Process

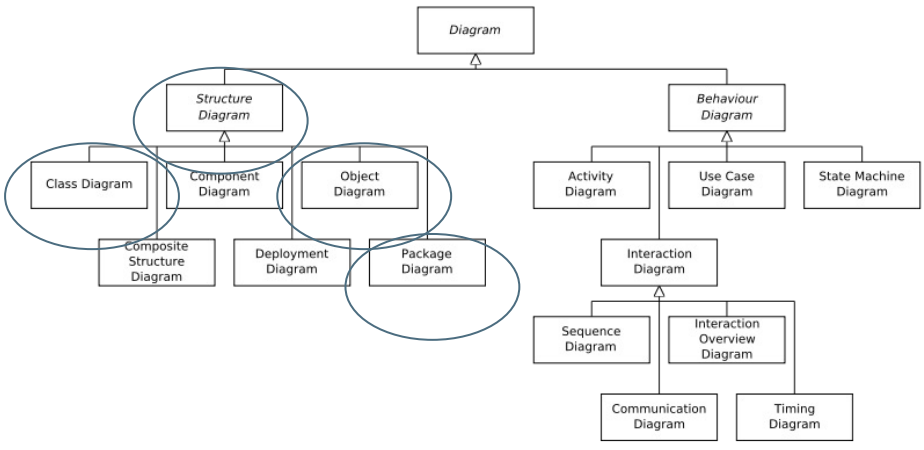
UNIFIED MODELING LANGUAGE

UML analysis, design, and implementation models



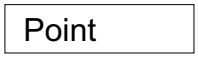
17

UML diagram overview



18

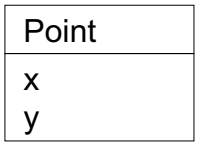
The class Point in UML



Análise

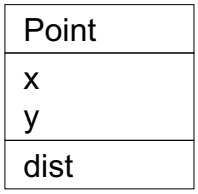
19

The class Point in UML

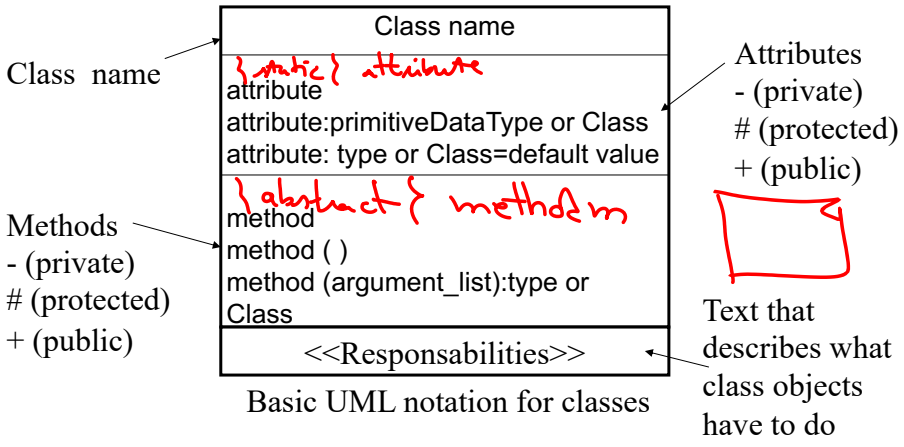


20

The class Point in UML



UML class

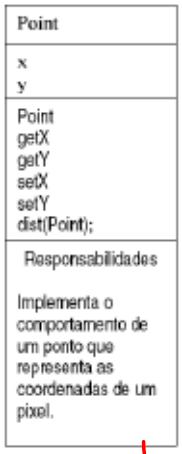


Basic UML notation for classes

UML class – example: class Point

```
class Point {
    private int x, y;

    public Point() { x=0; y=0;}
    public int getX() { return x; }
    public int getY() { return y; }
    public void setX(int x) { /* … */ }
    public void setY(int x) { /* … */ }
    public double dist(Point) { /* … */ }
}
```

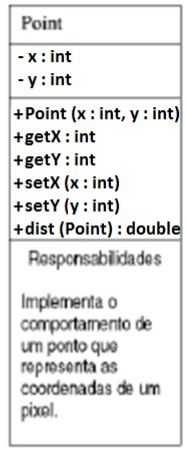


23

UML class – example: class Point

```
class Point {
    private int x, y;

    public Point() { x=0; y=0;}
    public int getX() { return x; }
    public int getY() { return y; }
    public void setX(int x) { /* … */ }
    public void setY(int x) { /* … */ }
    public double dist(Point) { /* … */ }
}
```



24

How to identify classes?

- There are very many techniques!
- Here's one of the simplest:

  Identify ***nouns*** in the problem description. For each noun ask:

  1. Is there any data that needs to be saved for this candidate?
  2. Is there any task that this candidate should perform?

  An affirmative answer to any of these questions elect the candidate to a class.

Typical classes

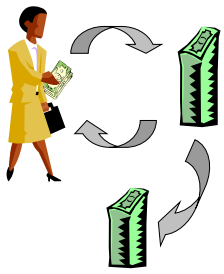
- Physical entities such as: car, place, person
- Concepts such as point, trajectory
- Roles such as teacher, employee, owner
- Events such as mouse click, exception

Canditates to discard immediately

- Synonymous that represent the same entity.
- Candidates out of the problem domain.
- Candidates that hold only single (or atomic) values – these will be *instance variables.*
- Candidates in the meta-language used to describe the problem.

P4A: A bank account

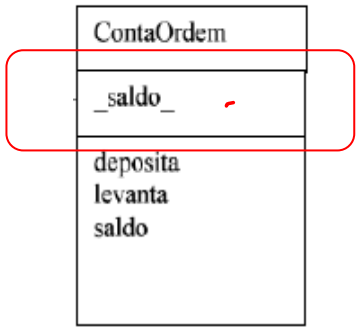
Develop a program for handling a bank account whose attributes include owner and a balance that cannot be negative



Identifying methods

- After you have a set of classes define the behavior (methods) of each class
- Look for ***verbs*** in the problem description
- Match the verbs to the appropriate classes

Bank accounts: a possible class



balance (saldo) holds just a primitive value
can be implemented by an instance variable

Relationships between classes: dependency

- A class `C` depends on class `S` if it uses objects of class `S` ("knows about" relationship)
- Example: `Client` depends on `Point` (to compute the distance between two points)



For the same functionality which organization is better? Why?

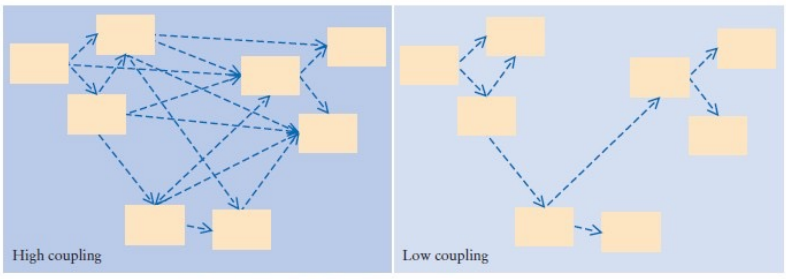
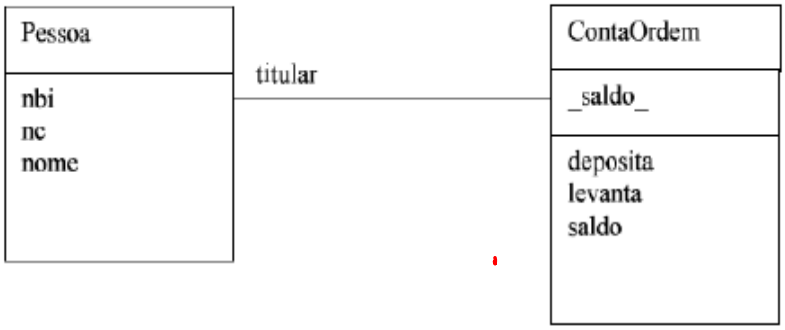


Figure 4 High and Low Coupling Between Classes

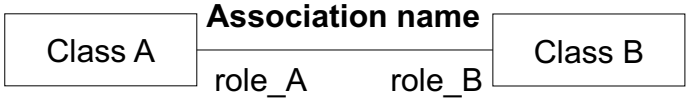
Bank accounts: a possible class diagram



Example of Association

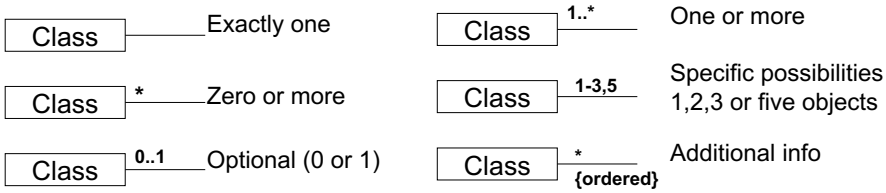
Associations

- Associations document relations between two or more classes at the same level of importance (or relevance)



Multiplicity of class relationships

- Multiplicity: refers to how many objects of one class can relate to each object of another class.

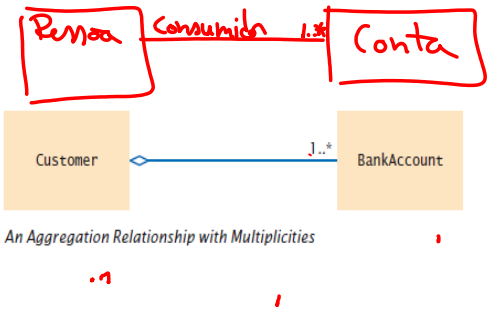


3-35

35

Example of multiplicity

- Multiplicity: refers to how many objects of one class can relate to each object of another class.

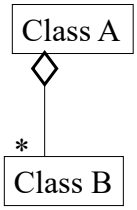


3-36

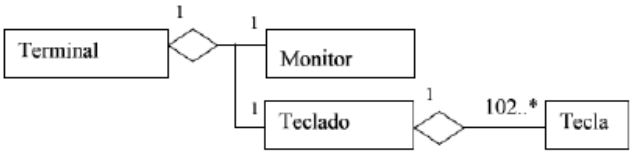
36

Aggregation

- Aggregation is a special type of association: a part-whole relationship – a class has an attribute which is an object of another class.



Example of Aggregation



Example of Aggregation

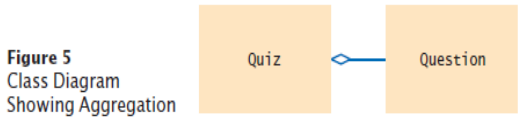
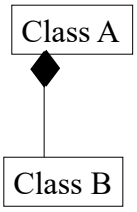


Figure 5
Class Diagram
Showing Aggregation

39

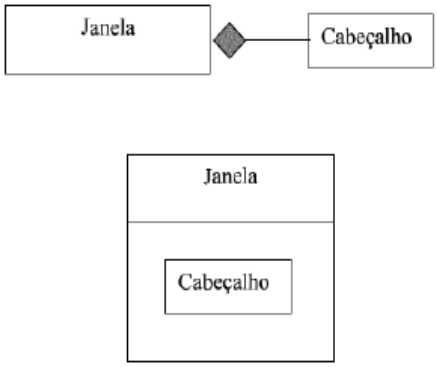
Composition

- Composition is a special case of aggregation where:
  - The class `A` is the only class that has objects of class `B`, as attributes
  - Class `A` creates and/or destroy objects of class `B`



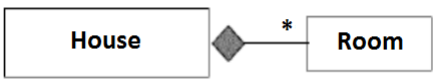
40

Example of composition



41

Composition, another example of



- A house can have several rooms
- If we destroy the house we also destroy the rooms

42

UML relationship symbols

Relationship	Symbol	Line Style	Arrow Tip
Inheritance	——▷	Solid	Triangle
Interface Implementation	- - - -▷	Dotted	Triangle
Aggregation	◇——	Solid	Diamond
Dependency	- - - ->	Dotted	Open

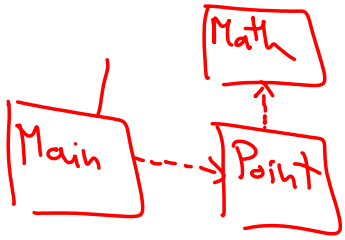
Para ver

X

X

Association

Composition



43

Learning outcomes

1. Learn how to discover new classes and methods
2. Identify association, aggregation, composition, dependency, and inheritance relationships between classes
3. Describe class relationships using UML class diagrams
4. Apply object-oriented design techniques to building complex programs
5. Understand the notion of design pattern
6. Understand and apply the The Expert pattern

44

Overview

- Modeling and UML, an Introduction
- Classes and relationship between classes: association, composition, and agregation
- Introduction to UML class and object diagrams
- Notion of *design pattern*
- The Expert pattern